

Thực hành phát triển hệ thống Trí tuệ nhân tạo RUP, Agile/XP

Trần Quốc Long

Trường ĐH Công nghệ, ĐHQG Hà Nội

Ngày 1 tháng 3 năm 2026

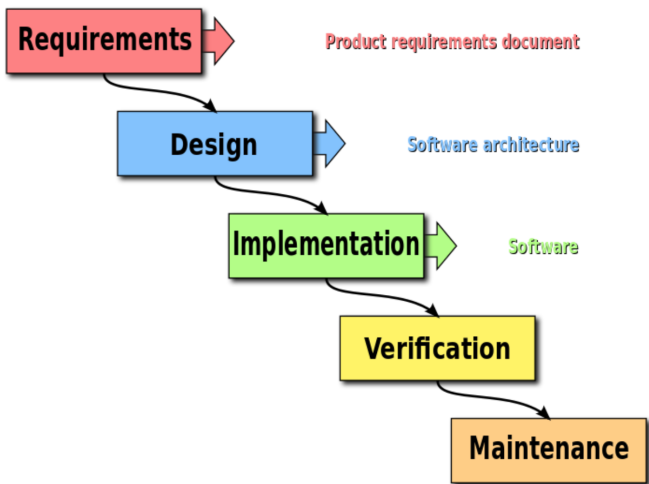
Mục lục

- 1 Các phương pháp phát triển phần mềm
- 2 RUP
- 3 Agile
- 4 Các yếu tố chất lượng của thiết kế

Các phương pháp phát triển

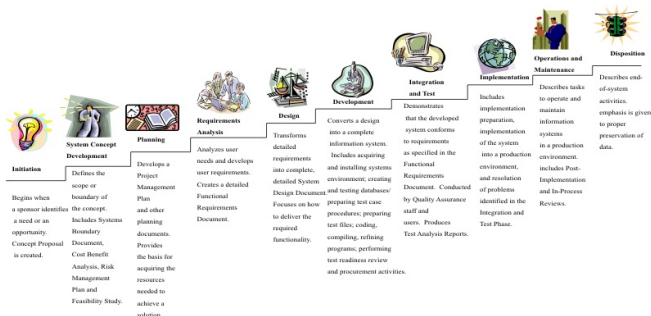
- Quy trình thác nước (Waterfall/SDLC)
- Quy trình phát triển thống nhất (RUP)
- Khung phần mềm của Microsoft (MSF)
- Thư viện cơ sở hạ tầng CNTT (ITIL)
- Phong cách Agile và các phương pháp khác

Quy trình thác nước



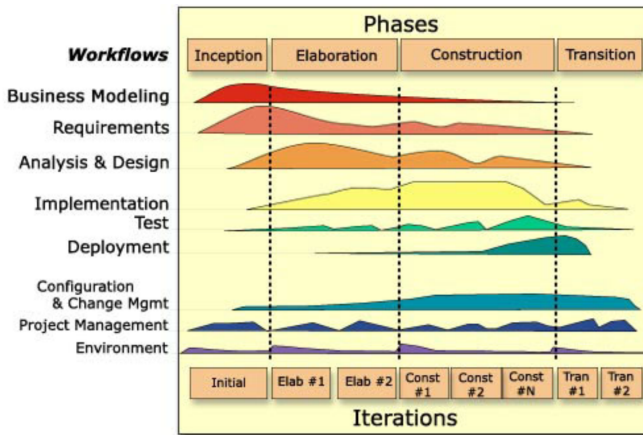
Vòng đời phần mềm (SDLC)

Systems Development Life Cycle (SDLC) Life-Cycle Phases

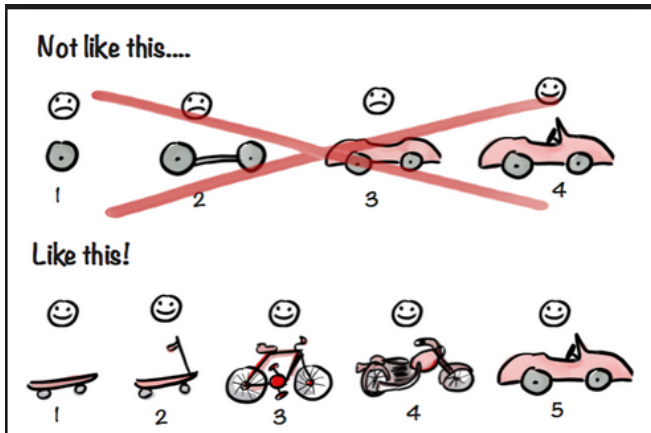


Quy trình phát triển thống nhất (RUP)

Lặp và tăng trưởng



Lặp (Iterative)



Tăng trưởng (Incremental)



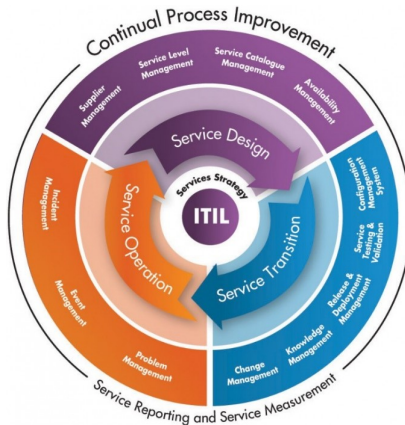
Vai trò trong RUP



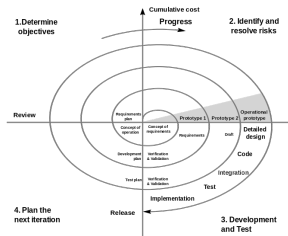
Khung phần mềm của Microsoft (MSF)



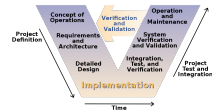
Thư viện cơ sở hạ tầng CNTT (ITIL)



Các phương pháp khác



Hình: Spiral



Hình: V-Model

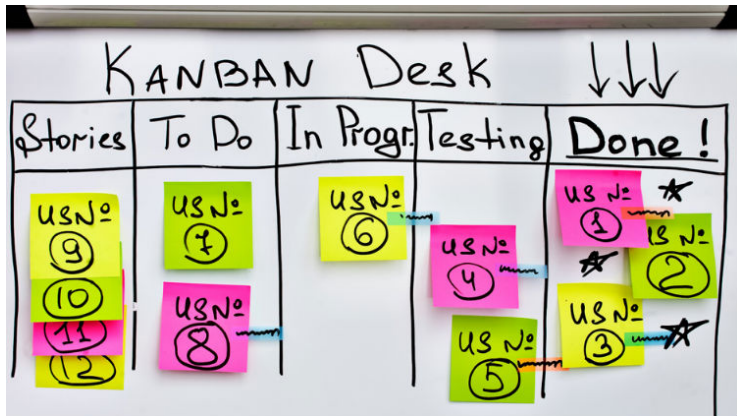


Hình: Rapid Application Development

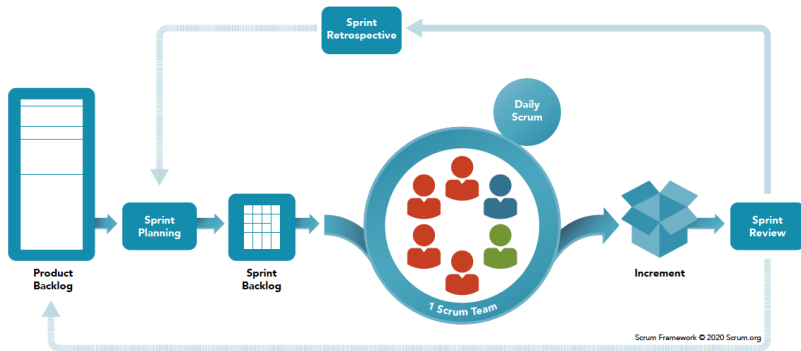
Quy trình Agile



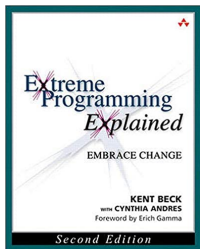
Công cụ Kanban



Khung Scrum



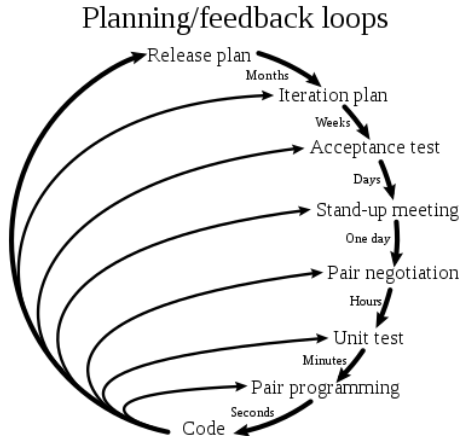
Lập trình cực đoan (XP)



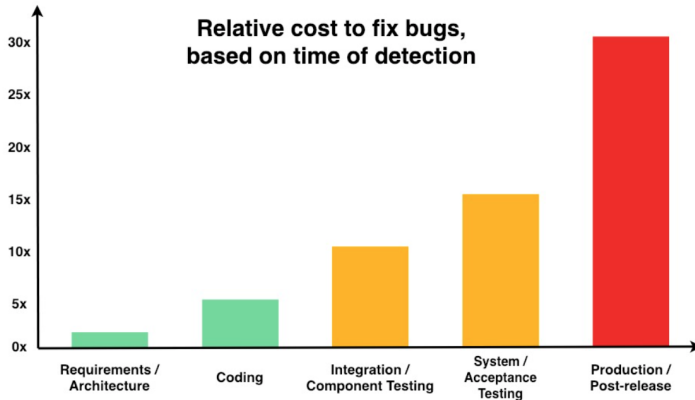
Liên tục tinh chỉnh thiết kế của hệ thống, bắt đầu từ một khởi đầu rất đơn giản. Loại bỏ bất kỳ sự linh hoạt nào không hiệu quả.

Kent Beck, *Extreme Programming Explained: Embrace Change*

Lập trình cực đoan (XP)



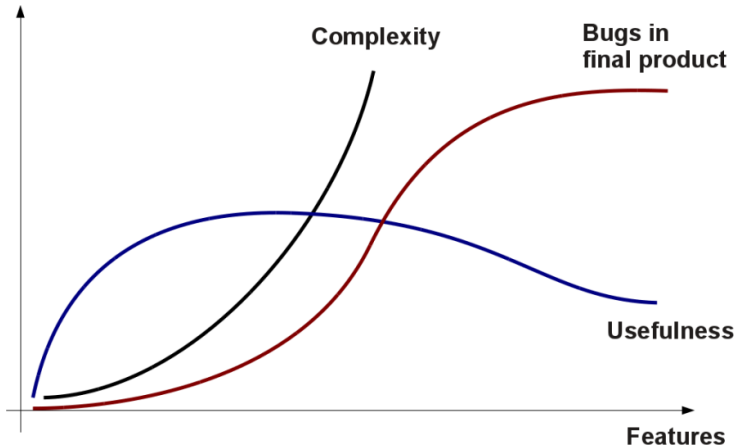
Chi phí của bug



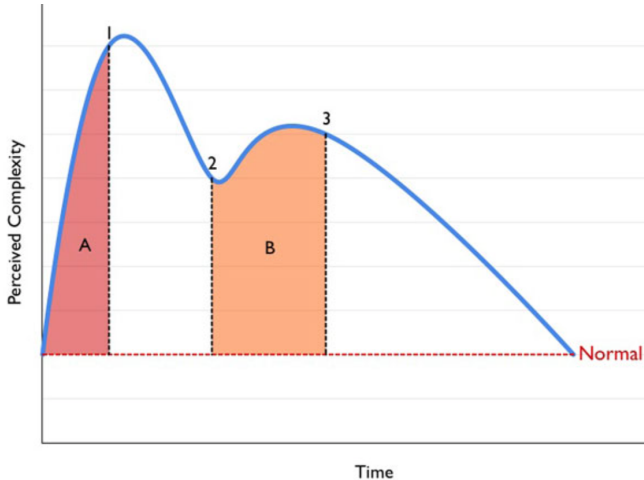
Tính đơn giản

- *Đầu tư* nhỏ nhất có thể vào thiết kế trước khi nhận được lợi nhuận từ nó. – **Kent Beck**
- Thiết kế chứa *sự phức tạp không cần thiết* khi có các yếu tố không hữu ích. Điều này thường xảy ra khi ta dự đoán các thay đổi có thể có đối với yêu cầu và tìm cách xử lý những thay đổi tiềm năng đó. – **Robert Martin**
- Không làm chủ sự phức tạp của phần mềm có hậu quả là các dự án bị trễ, vượt ngân sách và thiếu sót so với yêu cầu. Tình trạng này gọi là *khủng hoảng phần mềm*, nhưng thẳng thắn mà nói, một căn bệnh kéo dài như vậy phải được gọi là trạng thái bình thường. – **Grady Booch**

Tính đơn giản



Tính đơn giản



Tính đơn giản

Đức tính chính của một kiến trúc sư là khả năng *giảm bớt sự phức tạp*. Một kiến trúc sư giỏi không bao giờ tự hào về một sơ đồ phức tạp. Thay vào đó, họ sẽ tự hào về một bản vẽ đơn giản và dễ hiểu với vài hình chữ nhật mà giải thích hoàn hảo một ứng dụng đa tầng. Việc này thực sự khó. Đó là nơi mà tư duy kiến trúc thực sự tỏa sáng – **Yegor Bugayenko**

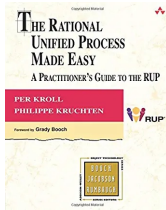
Tính không trùng lặp

- Khi có *mã dư thừa* trong hệ thống, việc thay đổi hệ thống trở nên khó khăn. Các lỗi được tìm thấy phải được sửa trong mọi lần lặp lại. Tuy nhiên, vì mỗi lần lặp lại hơi khác nhau, nên việc sửa lỗi không phải lúc nào cũng giống nhau – **Robert Martin**

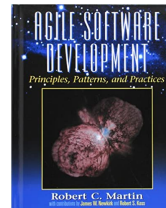
Tính mô-đun hóa

- Một thiết kế là *bất động* nếu nó chứa các thành phần được dùng trong các hệ thống khác, nhưng nỗ lực và rủi ro để tách các phần đó ra khỏi hệ thống ban đầu là quá lớn – **Robert Martin**
- Một thiết kế là *cứng* nếu một thay đổi đơn lẻ gây ra một loạt các thay đổi tiếp theo trong các module phụ thuộc. Càng nhiều module phải thay đổi, thiết kế càng *cứng* – **Robert Martin**

Tài liệu tham khảo

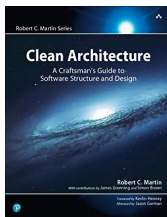


Hình: Per Kroll và cộng sự, *A Practitioner's Guide to the RUP*

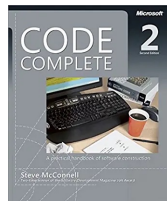


Hình: Robert Martin, *Agile Software Development. Principles, Patterns, and Practices*

Tài liệu tham khảo

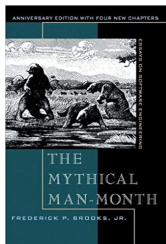


Hình: Robert Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*

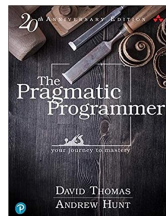


Hình: Steve McConnell, *Code Complete*

Tài liệu tham khảo



Hình: Frederick Brooks Jr., *Mythical Man-Month, The: Essays on Software Engineering*



Hình: David Thomas và cộng sự, *The Pragmatic Programmer: Your Journey To Mastery*